

מדריך למידה ומתודולוגיית פיתוח: PlaylistSample

המדריך המלא: מתודולוגיית עבודה (View מול ViewModel) ויישום מעשי ב-MVVM ב-.NET MAUI.

קהל יעד: מגמת הנדסת תוכנה / תיכון

נושא: מתודולוגיית MVVM למתחילים

פלטפורמה: .NET MAUI / NET 10

שלב: 1 מתוך מסלול אספים ומסדי נתונים

פרק 1: מתודולוגיית פיתוח – מאיפה מתחילים? View-First מול ViewModel-First

נניח שיש לנו רעיון לאפליקציה וכבר הגדרנו את מודל הנתונים של הסרטון (`VideoItem`). בנקודה זו כל תלמיד שואל את עצמו: **מאיפה להתחיל לקודד עכשיו?**

הגישה	היתרונות	המלכודות לתלמיד
גישת View-First (עיצוב המסך תחילה ב-XAML)	• משוב ויזואלי מיידי – רואים את התוצאה בעיניים מיד. • קל להגדיר מה צריך להיות במסך תוך כדי שרטוט פקדים.	• הפיתוי לשבור MVVM: דאבל-קליק על כפתור מוביל לכתיבת אירועי <code>Button_Clicked</code> בקוד האחורי. • Bindings "באוויר": חיבור למאפיינים שעדיין לא קיימים בקוד, ללא השלמה אוטומטית (IntelliSense).
גישת ViewModel-First (הגדרת המצב והפקודות תחילה)	• בדיקות מלאה (Testability): הלוגיקה נבדקת ופועלת לפני שנוגעים ב-XAML. • ה-XAML הופך להלבשה בלבד: כל ה-Bindings נשענים על מאפיינים אמיתיים ועובדים עם IntelliSense מלא.	• דורש יכולת הפשטה גבוהה – צריך לדמיין את התנהגות המסך מבלי לראות אותו עדיין גרפית.

"שביל הזהב" המומלץ לתלמידים: תהליך 4 השלבים 🏆

כדי לא לוותר על המשוב הוויזואלי ובו-בזמן לשמור על ארכיטקטורת MVVM תקינה, זהו סדר העבודה המנצח:

1 שרטוט מהיר על דף נייר (Wireframe) ב-2 דקות

מציירים מלבן פשוט של סמארטפון: איפה יושב סרגל הדפדוף? איפה שדות הקלט? ואיזה כפתורים קיימים? השרטוט עונה לנו מיד: מה המשתמש רואה ומה הוא עושה?

2 הגדרת המודל והשירות (Model & Service)

יוצרים את מחלקת הנתונים (`VideoItem`) ואת ממשק השירות (`IVideoService`) עם מתודות `Task` א-סינכרוניות. כעת יש לנו מקור מידע אמין.

3 מימוש ה-`ViewModel` (תרגום השרטוט ל-`State` ו-`Commands`)

- עוברים על השרטוט שציינו בשלב 1 וגוזרים:
- כל טקסט שמשתנה על המסך → הופך ל- `Property` עם `OnPropertyChanged()`.
 - כל כפתור שהמשתמש יכול ללחוץ → הופך ל- `Command` עם תנאי `CanExecute`.

4 הלבשת המסך ב-`XAML (View & Bindings)`

פותחים את קובץ ה-`XAML`, מניחים את הפקדים, ומבצעים `{Binding}` למאפיינים ולפקודות שכבר קיימים ובדקו ב-`ViewModel`.

דוגמה מוחשית: כפתור "איפוס/ניקוי טופס" 💡

בשרטוט המסך יש כפתור "נקה טופס".

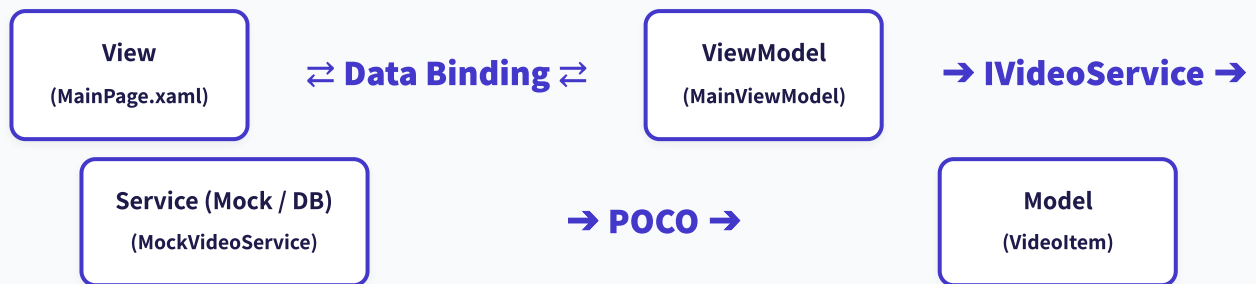
לפי

"שביל הזהב"

: בשלב 3 נגדיר ב-`ViewModel` פקודת `ClearFormCommand` אשר מאפסת את השדות `InputTitle`,
`InputUrl` וכו'. רק בשלב 4 נפתח את ה-`XAML` ונכתוב `Command="{Binding ClearFormCommand}"`. ה-`Binding` יתחבר מיד ללוגיקה שכבר עובדת!

פרק 2: יישום הלכה למעשה באפליקציית `PlaylistSample` 📺

תרשים זרימת הנתונים באפליקציה (Data Flow Architecture)



הפרדת אחריות מוחלטת: המסך אינו מכיר את מקור הנתונים, ושכבת השירות אינה מודעת לפקדי ה-UI.

הגדרת נקודת הכניסה לאפליקציה (ללא Shell)

שלב א

במקום להשתמש במנגנון Shell מסורבל, ב- `App.xaml.cs` אנו טוענים ישירות את דף הבית:

```
protected override Window CreateWindow(IActivationState? activationState)
{
    return new Window(new MainPage());
}
```

שכבת המודל: `Models/VideoItem.cs`

שלב ב

המודל הוא מחלקת POCO נקייה, מוכנה מראש לשדות מזהים של SQLite:

```
public class VideoItem
{
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public string Url { get; set; } = string.Empty;
    public string ChannelName { get; set; } = string.Empty;
    public int DurationMinutes { get; set; }
    public bool IsWatched { get; set; }

    public string FormattedDuration => DurationMinutes > 0 ? $"{DurationMinutes} דקות" : "0 דקות";
    public string WatchedStatusText => IsWatched ? "נצפה ✓" : "טרם נצפה ⌚";
}
```

שלבג שכתב השירות: חוזה א-סינכרוני ומימוש מוק סטטי

ממשק `IVideoService` מגדיר פעולות א-סינכרוניות (`Task`) מלאות:

```
public interface IVideoService
{
    Task<IReadOnlyList<VideoItem>> GetAllVideosAsync();
    Task<VideoItem?> GetVideoByIdAsync(int id);
    Task<bool> AddVideoAsync(VideoItem video);
    Task<bool> UpdateVideoAsync(VideoItem video);
    Task<bool> DeleteVideoAsync(int id);
}
```

השירות `MockVideoService` מממש את הממשק מעל רשימה סטטית בזיכרון בתוספת
.await Task.Delay(...)

שלבג שכתב ה-ViewModel: ניהול מצב, דפדוף ו-CanExecute

`MainViewModel` מנהל את האינדקס הנוכחי `_currentIndex`, מעדכן את הפריט הפעיל `CurrentVideo`,
ומפעיל את בדיקות ה-`CanExecute` של כפתורי הדפדוף:

```
// ViewModel-בנאי ה
NextCommand = new Command(ExecuteNext, CanExecuteNext);
PreviousCommand = new Command(ExecutePrevious, CanExecutePrevious);

private bool CanExecuteNext() => IsNotLoading && _videos.Count > 0 && _currentI
private bool CanExecutePrevious() => IsNotLoading && _videos.Count > 0 && _curr
```

שלב ה-3: שכתב המסך (View) וחיבור ה-DataContext

המסך ב- `MainPage.xaml` מכיל כרטיסיית דפדוף (עם כותרת, ערוץ, משך, כתובת URL וסטטוס) לצד כרטיסיית טופס הזנה. החיבור ב-Code-Behind ישיר ושקוף:

```
public partial class MainPage : ContentPage
{
    public MainPage()
    {
        InitializeComponent();
        // מורכב DI Container ללא ViewModel-חיבור ישיר ושקוף של ה
        DataContext = new MainViewModel(new MockVideoService());
    }
}
```

פרק 3: שאלות חשיבה ואתגרים מעשיים לתלמיד

שאלה 1: מנגנון CanExecute מול בדיקות ידניות

מדוע עדיף להשתמש ב- `CanExecute` של ה-Command שקובע האם הכפתור מאפשר או מושבת, במקום לבצע בדיקות ב-Code-behind כגון `button.IsEnabled = (index > 0);`?

שאלה 2: המעבר העתידי ל-SQLite

כאשר נחליט להחליף את `MockVideoService` במסד נתונים של SQLite, אילו קבצים בפרויקט נצטרך לערוך, ומדוע ה-ViewModel וה-XAML יישארו ללא שינוי?

שאלה 3: אתגר מעשי לתלמיד (כפתור מחיקה)

הוסיפו למסך כפתור "מחק סרטון נוכחי" (`DeleteCommand`).

- איזה תנאי תגדירו במתודת ה- `CanExecuteDelete` ?
- אילו פעולות תבצע מתודת ה- `ExecuteDeleteAsync` מול ה-Service ומול ה-ViewModel?